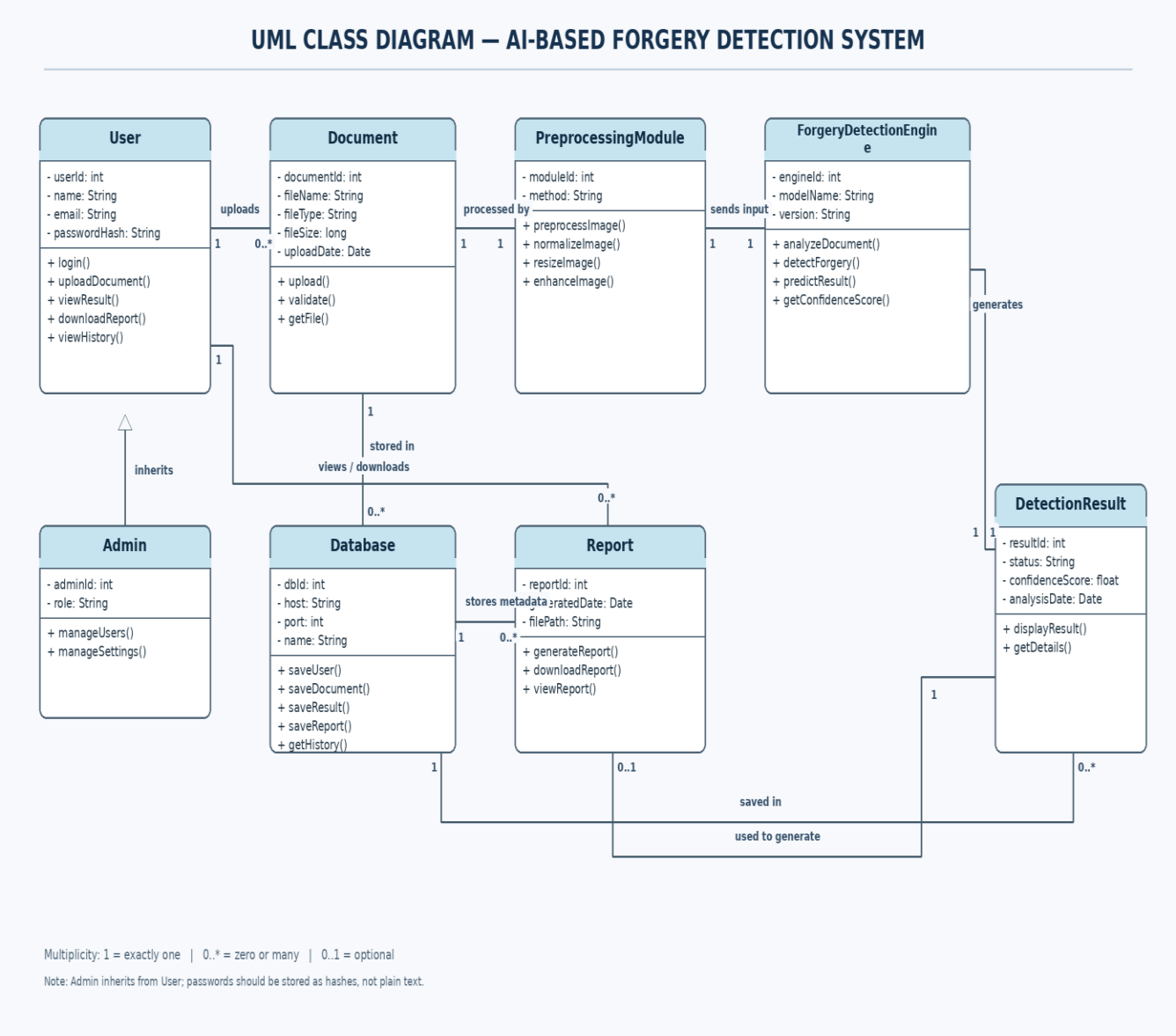
UML Class Diagram: AI-Based Forgery Detection System



1. Introduction

A UML class diagram shows the static structure of a software system: the classes it contains, the data and operations each class holds, and the relationships between them. The diagram studied here models an **AI-based forgery detection system**, in which a user uploads a document, the system analyses it with a trained model, and a result and report are returned to the user. The diagram has eight classes: User, Admin, Document, PreprocessingModule, ForgeryDetectionEngine, DetectionResult, Report and Database. Together they describe the full path of a document from upload to a stored, downloadable report.

2. Classes and Their Responsibilities

User. The main actor of the system. It holds userId, name, email and passwordHash, and provides the operations login(), uploadDocument(), viewResult(), downloadReport() and viewHistory(). Storing a password hash rather than the password itself is a sound security choice, as the note on the diagram also points out.

Admin. A specialised kind of User with its own attributes, adminId and role, and the operations manageUsers() and manageSettings(). It represents the person who maintains the system.

Document. Represents the file submitted for checking. Its attributes are documentId, fileName, fileType, fileSize and uploadDate, and its operations are upload(), validate() and getFile(). The validate() operation lets the system reject unsupported or invalid files before any analysis starts.

PreprocessingModule. Prepares the document for the model. It has a moduleId and a method attribute, and operations such as preprocessImage(), normalizeImage(), resizeImage() and enhanceImage(). Consistent size, scale and quality of the input usually improve the accuracy of the detection model.

ForgeryDetectionEngine. The core AI component. It stores engineId, modelName and version, which makes it possible to track which model produced a given result. Its operations are analyzeDocument(), detectForgery(), predictResult() and getConfidenceScore().

DetectionResult. Holds the outcome of one analysis: resultId, status, confidenceScore (a float) and analysisDate. It offers displayResult() and getDetails() so that the outcome can be shown to the user.

Report. A formal record built from a result. It has reportId, generatedDate and filePath, and the operations generateReport(), downloadReport() and viewReport().

Database. The storage layer. Its attributes are dbId, host, port and name, and its operations are saveUser(), saveDocument(), saveResult(), saveReport() and getHistory(). It keeps the data needed to show a user's history later.

3. Relationships Between Classes

The diagram uses association, multiplicity and inheritance to show how the classes work together.

Relationship	Multiplicity	Meaning
User uploads Document	1 to 0..*	One user can upload many documents,

Relationship	Multiplicity	Meaning
		or none yet.
Document is processed by PreprocessingModule	1 to 1	Each document passes through preprocessing.
PreprocessingModule sends input to ForgeryDetectionEngine	1 to 1	The cleaned data is passed on for analysis.
ForgeryDetectionEngine generates DetectionResult	1 to 1	Each analysis produces one result.
DetectionResult is used to generate Report	1 to 0..1	A report is optional for a result.
User views / downloads Report	1 to 0..*	A user can view or download many reports.
Documents, results and reports are stored in Database	1 to 0..*	The database holds many stored records.
Admin inherits from User	Generalisation	An admin has everything a user has, plus management functions.

The inheritance link between Admin and User is drawn with a hollow triangle, the standard UML symbol for generalisation. It avoids repeating login and related behaviour in two classes.

4. Flow of the System

Reading the diagram from left to right and then downwards gives the working flow of the system:

1. The user logs in and uploads a document.
2. The document is validated and passed to the PreprocessingModule, which normalises, resizes and enhances it.
3. The ForgeryDetectionEngine analyses the prepared document and predicts whether it is genuine or forged, along with a confidence score.
4. A DetectionResult object records the outcome.
5. If needed, a Report is generated from the result.
6. The Database saves the document, result and report, so the user can view the result, download the report and check history later.

5. Design Observations

The diagram separates concerns well. Preprocessing, detection, result handling and reporting are separate classes, so each can be improved or replaced without affecting the others; for example, a new model version can be plugged into the engine without changing the report logic. Recording modelName and version in the engine, and a confidence score in the result, also supports transparency and traceability of AI decisions.

Some points could be refined. The Database is shown as a class with direct links to several others, whereas in many designs it would be a separate data-access layer. The Document is also linked to one preprocessing module, though a real system might choose different preprocessing methods for different file types. These are design choices rather than errors, and they do not reduce the clarity of the model.

6. Conclusion

The class diagram gives a clear and well-organised blueprint of the forgery detection system. It identifies the key objects, their data and behaviour, and the way a document moves from upload through preprocessing and AI analysis to a stored result and report. The use of multiplicities, inheritance and meaningful association labels makes the model easy to read and a solid starting point for implementation.